02

MGU BCA - 3RD SEM

Design and Analysis of Algorithms

Module 2 - Divide and Conquer, Greedy

By Aben Sojan

Exam - 12 Nov 2026

Complete question bank: every important, repeated and sure-mark question for this module, each with a full-mark model answer. Nothing important left out.

EXAM PATTERN (from real MGU question papers)

DAA Honours exam: 50 marks, 1 hour 30 minutes. Part A: 10 x 2 marks, all compulsory, short questions from all modules. Part B: 3 x 5 marks, focused explanations with examples. Part C: 1 x 15 marks, full algorithm trace or essay. Total: 20 + 15 + 15 = 50. Part C clusters on MST (Prim's/Kruskal's), knapsack, Floyd-Warshall, N-Queens.

TOPIC 1

Divide and Conquer: General Concept

1

Three steps: DIVIDE the problem into smaller subproblems, CONQUER them recursively, COMBINE the results.

Subproblems are usually the same type and independent of each other.

Examples: merge sort, quick sort, binary search, Strassen's multiplication.

QUESTION BANK

1 Explain the divide and conquer strategy with its three steps.

SURE MARK (May 2019, Oct 2021, Feb 2025, Oct 2025 H)

Divide and conquer is an algorithm design strategy that solves a problem by breaking it into smaller copies of itself.

Step 1, DIVIDE: split the problem into smaller subproblems, usually of the same type and roughly half the size.

Step 2, CONQUER: solve the subproblems recursively; when a subproblem becomes tiny, solve it directly.

Step 3, COMBINE: merge the subproblem answers into the final answer for the original problem.

Example, merge sort: divide the array into two halves, conquer by sorting each half, combine by merging the two sorted halves.

The running time naturally becomes a recurrence, e.g. T(n) = 2T(n/2) + n, which solves to O(n log n).

Whenever a problem splits into independent smaller copies of itself, divide and conquer is the first strategy to try.

2 What are the advantages of divide and conquer?

IMPORTANT

Better complexity: splitting often drops the running time dramatically, e.g. O(n log n) sorting instead of O(n^2).

Parallelism: the subproblems are independent of each other, so they can be solved simultaneously on multiple processors.

Simple analysis: the recurrence T(n) = aT(n/b) + f(n) is solved mechanically by the Master method.

Cache friendliness: working on small chunks fits better into CPU caches, which speeds up real programs noticeably.

Elegant code: the recursive structure is short and mirrors the problem definition, so it is easy to write correctly.

These advantages explain why merge sort, quick sort, binary search and Strassen's algorithm are all divide-and-conquer algorithms.

3 Write the control abstraction of the divide and conquer method.

IMPORTANT (Oct 2025 H)

The control abstraction is the general recursive skeleton of divide and conquer, independent of any specific problem.

```
Algorithm DAndC(P)
// P is the problem instance, n its size
if Small(P) then return S(P)
else
divide P into P1, P2, ..., Pk, k >= 1
apply DAndC to each subproblem
return Combine(DAndC(P1), DAndC(P2), ..., DAndC(Pk))
```

Small(P) tests whether P is tiny enough to solve directly; S(P) solves such a tiny instance directly.

Divide splits P into k smaller instances of the same type; each is conquered by a recursive call.

Combine merges the subproblem answers into the answer for the original problem P.

Example: in merge sort, Small(P) is true for a one-element array, and Combine is the merge step.

In short: divide, conquer recursively, combine - every divide and conquer algorithm follows this skeleton.

TOPIC 2

Binary Search

2

Binary search finds a key in a SORTED array by repeatedly halving the search range.

Compare key with middle element: equal means found; smaller means search left half; larger means search right half.

Recurrence: T(n) = T(n/2) + 1, which solves to O(log n).

QUESTION BANK

4 Explain binary search with an example. What is its time complexity?

IMPORTANT

Binary search finds a key in a SORTED array by repeatedly throwing away the half that cannot contain the key.

Pseudocode:

```
low = 0; high = n - 1
while low <= high:
mid = (low + high) // 2
if a[mid] == key: return mid
elif key < a[mid]: high = mid - 1
else: low = mid + 1
return -1 # not found
```

Trace: search 23 in [5, 12, 23, 34, 45]. mid = index 2, a[2] = 23 = key, found immediately.

Longer trace: search 34. mid = 23 < 34, so low = 3; now mid = index 3, a[3] = 34, found.

Each comparison halves the search range, giving the recurrence T(n) = T(n/2) + 1.

Solving it: after k steps the size is n/2^k, which reaches 1 when k = log2(n), so T(n) = O(log n).

Space is O(1) for the loop version, since only a few variables are used.

Halving the problem at every step is what makes binary search exponentially faster than a linear scan.

5 **Why must the array be sorted for binary search?**

IMPORTANT

The whole trick of binary search is discarding half the array after a single comparison with the middle element.

That decision is valid only if everything left of the middle is smaller and everything right is larger - which is exactly what sorted order guarantees.

In an unsorted array the key could be hiding in the half we throw away, so the algorithm would wrongly report not found.

Therefore sorting is a precondition, not an optimisation: we sort once at O(n log n) and then enjoy O(log n) searches forever after.

Sorted order is the logical foundation the halving argument stands on; without it the algorithm is simply incorrect.

TOPIC 3

3

Finding Maximum and Minimum

Straightforward method: scan once, 2n - 2 comparisons in the worst case.

Divide and conquer: split array, find max/min of each half, compare the two results.

D&C needs only about 3n/2 - 2 comparisons, better than 2n - 2.

QUESTION BANK

6 **How many comparisons does the straightforward max-min algorithm need?**

SURE MARK (Oct 2021, Feb 2025, Oct 2025 H)

The straightforward method finds the maximum and the minimum in two separate scans of the array.

First scan: compare each new element with the current maximum - that takes n - 1 comparisons.

Second scan: compare each new element with the current minimum - another n - 1 comparisons.

Total in the worst case: (n - 1) + (n - 1) = 2n - 2 comparisons.

Example: for n = 100 numbers, that is 198 comparisons.

It is simple to write but wasteful, because every element is compared twice - once against the max and once against the min.

The divide-and-conquer version below cuts this to about 3n/2 - 2 by comparing elements with

each other first.

7 **Explain the divide and conquer approach for max and min.**

SURE MARK (Oct 2021, Feb 2025, Oct 2025 H)

Idea: instead of comparing every element against the running max and min, first pair elements up and compare them with each other.

Divide the array into two halves; recursively find (max1, min1) of the left half and (max2, min2) of the right half.

```
overall max = larger of max1 and max2 # 1 comparison
overall min = smaller of min1 and min2 # 1 comparison
```

Combining two halves costs only 2 comparisons, giving the recurrence T(n) = 2T(n/2) + 2.

This solves to about 3n/2 - 2 comparisons, clearly better than the straightforward 2n - 2.

Example: for n = 100, divide and conquer needs about 148 comparisons versus 198 for the naive method.

The saving comes from letting elements eliminate each other before facing the current champions.

8 **For an array of 6 elements, compare the number of comparisons needed to find both the maximum and minimum using linear search and divide and conquer.**

SURE MARK (Oct 2021, Feb 2025, Oct 2025 H)

Linear search (straightforward method): compare each new element with the current max and the current min - 2 comparisons per element after the first.

Comparisons = 2n - 2 = 2(6) - 2 = 10 comparisons.

Divide and conquer (pair method): pair up the 6 elements - 3 comparisons produce 3 pair-winners and 3 pair-losers.

Find the max of the 3 winners: 2 comparisons. Find the min of the 3 losers: 2 comparisons.

Total = 3 + 2 + 2 = 7 comparisons.

So divide and conquer needs 7 comparisons against 10 for linear search - 3 fewer for the same answer.

Answer: linear search 10 comparisons, divide and conquer 7 comparisons.

TOPIC 4

4

Merge Sort

Divide the array into halves, sort each half recursively, then merge the two sorted halves.

Merge step is linear: compare front elements and take the smaller, O(n).

Recurrence T(n) = 2T(n/2) + n gives O(n log n) in best, average AND worst case.

Stable sort, but needs O(n) extra space; not in-place.

QUESTION BANK

9 **Explain merge sort with an example and analyse its complexity.**

SURE MARK (May 2019, Oct 2021, Feb 2025)

Merge sort is the classic divide-and-conquer sort: split the array into halves, sort each half recursively, then merge the sorted halves.

Pseudocode:

```
mergesort(a):
if len(a) <= 1: return a
mid = len(a) // 2
left = mergesort(a[:mid]); right = mergesort(a[mid:])
return merge(left, right) # linear scan taking the smaller front element
```

Trace on [38, 27, 43, 3]: split into [38, 27] and [43, 3]; these sort to [27, 38] and [3, 43].

Merge step: compare 27 vs 3, take 3; 27 vs 43, take 27; 38 vs 43, take 38; then 43.

Result: [3, 27, 38, 43].

Recurrence: two halves plus a linear merge gives T(n) = 2T(n/2) + n, which solves to O(n log n).

Because the split is always exactly half, this bound holds for the best, average AND worst case alike.

Space cost: O(n) extra for the merge buffers.

Merge sort is also stable: equal elements keep their original relative order.

Guaranteed O(n log n) in every case is why merge sort is the safe choice for sorting.

10 Why is merge sort O(n log n) even in the worst case?

SURE MARK (May 2019, Oct 2021, Feb 2025)

The split step always cuts the array into two exactly equal halves, no matter what the data looks like.

So the recursion depth is always log2(n); no input can make the tree deeper or shallower.

At every level, the merge step scans all n elements once across the subproblems, doing n total work per level.

Total work = (work per level) x (number of levels) = n x log n, for every possible input without exception.

Contrast quick sort: its split depends on the pivot, so a bad pivot gives an unbalanced tree and O(n^2).

Merge sort's perfectly balanced partition is built into the algorithm, so the worst case cannot escape O(n log n).

11 What are the drawbacks of merge sort?

IMPORTANT

Extra memory: merging needs O(n) auxiliary space, while quick sort and heap sort work in place.

Recursion overhead: for tiny subarrays the function-call cost exceeds the sorting work, making it slower than insertion sort there.

Not adaptive: it does the full n log n work even on an already sorted array, unlike insertion sort's O(n) best case.

In practice libraries fix this with a hybrid: use merge sort down to small sizes, then finish with insertion sort.

So merge sort trades memory for a rock-solid time guarantee - the right call when memory is available.

12 **Illustrate the recursion tree of merge sort for 10 elements.**

IMPORTANT (Oct 2021)

The recursion tree of merge sort shows how the array keeps splitting until single elements, then merges back up.

Take [38, 27, 43, 3, 9, 82, 10, 5, 21, 16] (10 elements).

Level 0: one node holding all 10 elements.

Level 1: two nodes of 5: [38, 27, 43, 3, 9] and [82, 10, 5, 21, 16].

Level 2: four nodes of sizes 3, 2, 3, 2: [38, 27, 43], [3, 9], [82, 10, 5], [21, 16].

Level 3: [38, 27], [43], [3], [9] and [82, 10], [5], [21], [16].

Level 4: ten single-element leaves - the base case, each already sorted.

Merging climbs back up: every level merges n elements in total, and there are ceil(log2 10) = 4 levels, so the total work is O(n log n).

In short: the tree has 4 levels above the leaves and shows exactly why merge sort costs O(n log n).

TOPIC 5

Quick Sort

5

Pick a pivot, PARTITION so smaller elements go left and larger go right, then sort both sides recursively.

Best and average case: pivot splits evenly, T(n) = 2T(n/2) + n = O(n log n).

Worst case: pivot is always smallest or largest (e.g. sorted array, first element as pivot), T(n) = T(n-1) + n = O(n^2).

In-place and fast in practice; not stable.

QUESTION BANK

13 Explain quick sort partitioning with an example.

SURE MARK (May 2019, Oct 2021, Feb 2025)

Quick sort picks one element as the pivot, partitions the array so smaller elements go left and larger go right, then sorts both sides recursively.

Lomuto partition pseudocode:

```
pivot = a[low]; i = low
for j in range(low+1, high+1):
if a[j] < pivot: i += 1; swap(a[i], a[j])
swap(a[low], a[i]) # pivot into its final position; return i
```

Trace on [10, 7, 8, 9, 1, 5] with pivot 10: every smaller element is swept left, giving [7, 8, 9, 1, 5, 10].

The pivot 10 is now fixed at the end and never moves again; recurse on [7, 8, 9, 1, 5].

Each partition scans its segment once, costing O(n) for the combine step of that level.

Since every element becomes a pivot exactly once, each is placed into its final position exactly once.

Partitioning is the heart of quick sort - everything else is just recursion on the two sides.

14 Analyse the best and worst cases of quick sort.

SURE MARK (May 2019, Oct 2021, Feb 2025)

The behaviour of quick sort depends entirely on where the pivot lands after partitioning.

Best case: the pivot always lands in the middle, splitting the array into two equal halves.

`Recurrence: T(n) = 2T(n/2) + n (partitioning costs n), which solves to O(n log n).`

Worst case: the pivot is always the smallest or largest element, so one side is empty and the other holds n-1 elements.

`Recurrence: T(n) = T(n-1) + n = n + (n-1) + ... + 1 = O(n^2).`

This worst case strikes with an already sorted array when the first element is always chosen as pivot.

The average case over random inputs is O(n log n), which is why quick sort is fast in practice despite the bad worst case.

In short: balanced pivots give n log n, degenerate pivots give n^2 - pivot choice is everything.

15 Compare merge sort and quick sort.

REPEATED (Oct 2021, Feb 2025)

Both are divide-and-conquer sorting algorithms with O(n log n) average time, but they differ in guarantees and resources.

Time: merge sort is O(n log n) in best, average and worst case; quick sort is O(n log n) on average but O(n^2) in the worst case.

Space: merge sort needs O(n) extra memory for merging; quick sort sorts in place using only O(log n) recursion stack.

Stability: merge sort is stable (equal elements keep their order); quick sort's swapping makes it unstable.

Practical speed: quick sort usually wins because of smaller constants and better cache behaviour from in-place access.

Choice rule: need a guaranteed time bound or stability - merge sort; need speed and low memory - quick sort.

This comparison is a favourite exam question because it tests the whole divide-and-conquer chapter at once.

16 How can the quick sort worst case be avoided?

IMPORTANT

The worst case comes from bad pivots, so the fix is to choose pivots that are unlikely to be extreme.

Random pivot: pick the pivot randomly instead of always the first element; an adversary cannot then feed a killer input, and the expected time stays O(n log n).

Median-of-three: take the median of the first, middle and last elements as the pivot; this kills the sorted-input worst case completely.

Hybrid approach: switch to insertion sort for tiny subarrays (e.g. below 10 elements) to cut recursion overhead.

Introspective sort, used in the C++ STL, goes further: it watches the recursion depth and switches to heap sort if it grows too deep, guaranteeing O(n log n).

With a decent pivot rule, the O(n^2) case becomes a theoretical curiosity rather than a practical danger.

TOPIC 6

6

Strassen's Matrix Multiplication

Normal multiplication of two n x n matrices needs 8 half-size multiplications: O(n^3).

Strassen uses 7 clever multiplications (M1 to M7) plus additions, trading 1 multiplication for extra additions.

Recurrence T(n) = 7T(n/2) + O(n^2), giving O(n^log2(7)) = O(n^2.81), better than n^3.

Best for large matrices; overhead makes it slower for small ones.

QUESTION BANK

17 **Explain Strassen's matrix multiplication. Why is it faster?**

SURE MARK (Oct 2021, Feb 2025, Oct 2025 H)

Standard matrix multiplication of two n x n matrices needs O(n^3) operations; a plain divide-and-conquer version needs 8 half-size multiplications.

Strassen's insight: with clever additions and subtractions of the submatrices, only 7 multiplications (M1 to M7) are enough to build all four result blocks.

One expensive multiplication is traded for several cheap additions, and multiplication dominates the cost.

Recurrence: 7 subproblems of size n/2 plus O(n^2) additions gives T(n) = 7T(n/2) + O(n^2).

By the Master method this solves to O(n^2.81), since log2(7) is about 2.81, which beats n^3 for large n.

Example: for n = 1024, n^3 is about 10^9 operations while n^2.81 is roughly 3 x 10^8 - a 3x saving.

Strassen proved that matrix multiplication is fundamentally easier than cubic, opening the whole field of fast matrix algorithms.

18 **Write the recurrence for Strassen's algorithm and solve it.**

SURE MARK (Oct 2021, Feb 2025, Oct 2025 H)

At each level Strassen creates 7 subproblems of half the size and does O(n^2) additions and subtractions to combine them.

`Recurrence: T(n) = 7T(n/2) + c*n^2, with T(1) = O(1).`

Apply the Master method: a = 7, b = 2, f(n) = n^2.

Compare n^(log_b a) = n^(log2 7) = n^2.81 against f(n) = n^2.

Since n^2.81 grows polynomially faster than n^2, we are in Master case 1: the subproblems dominate the cost.

Therefore T(n) = Theta(n^(log2 7)) = Theta(n^2.81).

The exponent drops from 3 to 2.81 purely because 7 multiplications replaced 8.

19 **What are the limitations of Strassen's algorithm?**

IMPORTANT

High constant factors: the many additions, subtractions and complex bookkeeping make each recursion level expensive.

Crossover point: it only beats the naive O(n^3) method for large matrices, typically n above a few hundred.

Extra memory: the intermediate sum and difference matrices need additional storage beyond the inputs.

Numerical stability: more additions mean more rounding-error accumulation in floating-point arithmetic.

Implementation complexity: the code is far harder to write and debug than three nested loops.

So Strassen is a theoretical breakthrough that libraries use selectively, not a drop-in replacement for everyday multiplication.

TOPIC 7

7

Greedy Method: General Concept

Greedy builds the answer step by step, always picking the locally best choice, never reconsidering.

Works only when local optimum leads to global optimum (greedy-choice property plus optimal substructure).

Examples: fractional knapsack, Prim's, Kruskal's, Dijkstra's, Huffman coding.

Fails on 0/1 knapsack: a greedy pick can block the true optimum.

QUESTION BANK

20 Explain the greedy strategy. When does it work?

SURE MARK (May 2019, Oct 2021, Feb 2025, Oct 2025 H)

The greedy strategy builds an answer step by step, always taking the choice that looks best right now, and never reconsidering earlier choices.

It is simple and fast, but it only works for problems with two structural properties.

Greedy-choice property: a globally optimal answer can be assembled from locally optimal choices - the first greedy pick is always part of some optimum.

Optimal substructure: after making the greedy choice, the remaining problem is a smaller copy of the original.

Example where it works: coin change with coins 25, 10, 5, 1 - repeatedly taking the largest coin that fits always uses the fewest coins.

Syllabus examples that are all greedy: fractional knapsack, Prim's, Kruskal's and Dijkstra's algorithms.

Greedy is take the best now, never look back - correct only when the problem's structure forgives short-sightedness.

21 Define feasible solution and optimal solution.

REPEATED (Oct 2021, Feb 2025)

A feasible solution is any candidate answer that satisfies all the constraints of the problem, whether or not it is the best one.

An optimal solution is the feasible solution with the best objective value - maximum profit or minimum cost.

Example, knapsack with capacity 10: item A (weight 6, profit 12) and item B (weight 5, profit 9).

Picking only A is feasible (weight 6 <= 10) with profit 12; picking only B is feasible with profit 9.

Picking A + B together is not feasible (weight 11 > 10).

The optimal solution is picking A alone, profit 12 - the best among all feasible picks.

In short: feasible means allowed, optimal means allowed and best.

22 Give an example where the greedy method fails.

IMPORTANT

The classic failure is the 0/1 knapsack, where items cannot be split.

Items: A (profit 60, weight 10), B (100, 20), C (120, 30); capacity 50.

Greedy by profit-to-weight ratio picks A first (ratio 6), then B (ratio 5): total weight 30, profit 160, and C no longer fits.

The true optimum takes B and C: weight 50, profit 220, which greedy never finds because it committed to A.

The failure happens because the greedy-choice property is missing: the locally best item A is not part of the global optimum.

Note the contrast: fractional knapsack allows taking part of C, and there greedy is optimal.

This example is the standard exam proof that greedy needs the right problem structure.

23 Write and explain the control abstraction of the greedy method.

IMPORTANT (Feb 2025)

The control abstraction is the general skeleton that every greedy algorithm follows.

```
Algorithm Greedy(a, n)
// a[1:n] holds the n inputs
solution = empty
for i = 1 to n do
x = Select(a)
if Feasible(solution, x) then
solution = Union(solution, x)
return solution
```

Select(a) picks the remaining input that looks best by the greedy criterion, e.g. the highest profit per weight.

Feasible(solution, x) checks that adding x keeps a valid partial answer, e.g. the total weight stays within capacity.

Union(solution, x) adds x to the answer; a greedy choice is never reconsidered later.

Example: fractional knapsack - Select picks the highest-ratio item, Feasible checks the capacity, Union adds it or a fraction of it.

In short: repeatedly select the best-looking feasible input - simple and fast, but correct only when the greedy-choice property holds.

24 What is subset paradigm?

IMPORTANT (Oct 2021)

Greedy problems fall into two paradigms: the subset paradigm and the ordering paradigm.

Subset paradigm: the answer is a subset of the n inputs - we decide for each input whether to take it or not.

Examples: knapsack (which items to pack), Prim's and Kruskal's (which edges form the minimum spanning tree).

Ordering paradigm: the answer is an ordering of the inputs, e.g. scheduling jobs in some sequence.

All the greedy problems in this syllabus - knapsack, Prim's, Kruskal's, Dijkstra's - are subset paradigm: each step picks one more input into the solution set.

In short: in the subset paradigm the answer is built as a set of chosen inputs.

TOPIC 8

Knapsack Problem

8

Knapsack: n items with profit p and weight w, capacity M; maximise profit without exceeding M.

Fractional knapsack: items can be split; greedy by profit/weight ratio is optimal, O(n log n).

0/1 knapsack: items taken whole or not at all; greedy fails, needs dynamic programming.

QUESTION BANK

25 Solve fractional knapsack with the greedy method. Give steps.

SURE MARK (May 2019, Oct 2021, Feb 2025, Oct 2025 H)

In the fractional knapsack, items may be split, and the goal is maximum profit within capacity M.

Step 1: compute the profit-to-weight ratio p/w for every item.

Step 2: sort items by ratio in descending order - this costs O(n log n).

Step 3: walk down the sorted list; take each whole item if it fits, otherwise take exactly the fraction that fills the knapsack, then stop.

Worked example: M = 50; items A (60, 10), B (100, 20), C (120, 30); ratios 6, 5, 4.

```
Take all of A: weight 10, profit 60. Remaining capacity 40.
Take all of B: weight 20, profit 100. Remaining capacity 20.
Take 20/30 of C: profit (20/30)*120 = 80. Knapsack full.
```

Total profit = 60 + 100 + 80 = 240, and this is provably optimal.

Optimality argument: any optimum can be rearranged to take items in ratio order without losing

profit, so greedy matches it.

Complexity: O(n log n) for sorting plus O(n) for the scan.

Divisibility is what makes the greedy choice safe here, unlike the 0/1 version.

26 **Differentiate fractional and 0/1 knapsack.**

SURE MARK (May 2019, Oct 2021, Feb 2025, Oct 2025 H)

The difference is divisibility: fractional knapsack allows taking part of an item; 0/1 knapsack forces take-it-or-leave-it.

Fractional: the greedy algorithm sorting by profit-to-weight ratio is optimal, running in O(n log n).

0/1: greedy can fail (the 60/100/120 example), so dynamic programming over capacity is needed, costing O(n*M).

Fractional is solvable in polynomial time; 0/1 knapsack is NP-hard in general.

Example: with capacity 50 and items (60,10), (100,20), (120,30), fractional gives 240 while 0/1 gives 220 by a different choice.

Memory aid: fractional means greedy works; zero-one means DP is needed.

One word - divisible or not - decides the entire algorithm.

27 **Solve the fractional knapsack instance with weights (10, 20, 30, 25, 28), profits (60, 100, 120, 75, 140) and capacity 50 using the greedy approach.**

SURE MARK (May 2019, Oct 2021, Feb 2025, Oct 2025 H)

First compute the profit-to-weight ratio of each item.

Item 1: 60/10 = 6.0; Item 2: 100/20 = 5.0; Item 3: 120/30 = 4.0; Item 4: 75/25 = 3.0; Item 5: 140/28 = 5.0.

Sort by ratio in descending order: Item 1 (6.0), Item 2 (5.0), Item 5 (5.0), Item 3 (4.0), Item 4 (3.0).

Capacity 50. Take Item 1 fully: weight 10, profit 60, remaining capacity 40.

Take Item 2 fully: weight 20, profit 100, remaining capacity 20.

Item 5 weighs 28 but only 20 fits: take the fraction 20/28, profit 140 x 20/28 = 100, remaining capacity 0.

Total profit = 60 + 100 + 100 = 260.

Answer: maximum profit 260, by taking items 1 and 2 fully and 20/28 of item 5.

TOPIC 9

Prim's Algorithm

9

Prim's finds the Minimum Spanning Tree: cheapest set of edges connecting all vertices, no cycles.

Grow one tree: start anywhere, repeatedly add the cheapest edge joining the tree to an outside vertex.

Uses a greedy choice; with a simple array it is O(V^2), with a heap O(E log V).

QUESTION BANK

28 **Explain Prim's algorithm step by step with an example.**

SURE MARK (May 2019, Oct 2021, Feb 2025, Oct 2025 H)

Prim's algorithm finds the Minimum Spanning Tree: the cheapest set of edges connecting all vertices without forming cycles.

It grows a single tree, starting from any vertex, always attaching the cheapest edge that reaches outside the tree.

Pseudocode:

```
tree = {start}; repeat V-1 times:
pick the minimum-weight edge (u, v) with u in tree, v outside
add v and the edge to the tree
```

Trace on triangle A-B 1, B-C 2, A-C 3, starting at A.

```
Tree = {A}. Edges from the tree outward: A-B (1), A-C (3). Pick A-B.
Tree = {A, B}. Edges outward: A-C (3), B-C (2). Pick B-C.
Tree = {A, B, C}. Done. MST edges: A-B, B-C with total cost 3.
```

Note A-C (3) was never taken: it would close the cycle A-B-C-A.

Complexity: O(V^2) with a simple array scanning for the minimum; O(E log V) with a min-heap.

The greedy choice is safe by the cut property: the lightest edge crossing any cut belongs to some MST.

Prim's is Dijkstra-like growth, but minimising edge weight instead of distance from the source.

29 **Why does the greedy choice work in Prim's algorithm?**

SURE MARK (May 2019, Oct 2021, Feb 2025, Oct 2025 H)

The justification is the cut property of minimum spanning trees.

Cut property: for any partition of the vertices into two sets, the lightest edge crossing between them belongs to at least one MST.

Proof sketch: if some MST avoided that lightest edge, adding it creates a cycle; removing the heaviest edge of that cycle gives another spanning tree that is no costlier - so the lightest edge is safe.

At every step, Prim's current tree defines exactly such a cut: vertices inside the tree versus outside.

The algorithm picks the lightest edge crossing this cut, which the property guarantees is safe to include.

Since every added edge is safe, the finished tree of V-1 edges must be a minimum spanning tree.

The cut property is the one-line proof that Prim's greedy choices never go wrong.

TOPIC 10

10

Kruskal's Algorithm

Kruskal's also finds the MST: sort ALL edges by weight, add the cheapest one that does not form a cycle.

Cycle check uses disjoint sets (union-find): skip an edge if its ends are already connected.

Complexity dominated by sorting: O(E log E).

QUESTION BANK

30 **Explain Kruskal's algorithm step by step with an example.**

SURE MARK (May 2019, Oct 2021, Feb 2025, Oct 2025 H)

Kruskal's finds the MST by considering all edges globally, cheapest first, skipping any edge that would form a cycle.

Pseudocode:

```
sort all edges by weight ascending
make a separate set for each vertex
for each edge (u, v) in sorted order:
if find(u) != find(v): add edge to MST; union(u, v)
```

```
else: skip it # would create a cycle
```

Trace on edges AB 1, BC 2, AC 3.

```
Sorted: AB(1), BC(2), AC(3). Take AB: sets become {A,B}, {C}.
Take BC: find(B) != find(C), so merge into {A,B,C}. MST so far: AB, BC.
AC: find(A) == find(C), both in {A,B,C} - skip, it would close a cycle.
```

MST cost = 1 + 2 = 3.

Cycle detection uses disjoint-set union (union-find), nearly O(1) per operation with path compression and union by rank.

Complexity is dominated by sorting: O(E log E).

Sort globally, take greedily, skip cycles - that is the whole algorithm.

31 Compare Prim's and Kruskal's algorithms.

IMPORTANT

Both are greedy algorithms that produce a minimum spanning tree of the same minimum total cost.

Growth style: Prim's grows a single tree outward from a start vertex; Kruskal's merges a forest by scanning a globally sorted edge list.

Data needed: Prim's works from the adjacency structure of the graph; Kruskal's works naturally from a plain edge list.

Complexity: Prim's is O(V^2) in the simple version or O(E log V) with a heap; Kruskal's is O(E log E), dominated by sorting.

Dense graphs (many edges): Prim's O(V^2) wins.

Sparse graphs (few edges): Kruskal's O(E log E) wins.

Implementation: Prim's needs a priority queue of frontier vertices; Kruskal's needs union-find for cycle checks.

Memory aid: Prim grows, Kruskal sorts.

Same answer, different strategy - pick based on how dense the graph is.

TOPIC 11

Dijkstra's Single Source Shortest Path

11

Dijkstra finds shortest paths from one source to all vertices in a graph with non-negative weights.

Greedy: keep dist[] (best known distance); repeatedly finalise the unsettled vertex with smallest dist

and relax its edges.

Relaxation: if dist[u] + w(u,v) < dist[v], update dist[v].

O(V^2) simple, O(E log V) with heap; FAILS with negative edge weights.

QUESTION BANK

32 **Explain Dijkstra's algorithm step by step with an example.**

SURE MARK (May 2019, Oct 2021, Oct 2025 H)

Dijkstra's algorithm finds the shortest paths from a single source to all vertices, requiring non-negative edge weights.

It maintains dist[], the best known distance to each vertex, and repeatedly finalises the closest unsettled vertex.

Pseudocode:

```
dist[source] = 0; dist[others] = infinity; settled = empty
repeat V times:
u = unsettled vertex with smallest dist; mark u settled
for each neighbour v of u: dist[v] = min(dist[v], dist[u] + w(u,v)) #
relaxation
```

Trace: edges S-A 4, S-B 2, B-A 1.

Start: dist = {S:0, A:inf, B:inf}.

```
Settle S (0): relax neighbours -> A = 4, B = 2.
Settle B (2): relax A -> min(4, 2+1) = 3.
Settle A (3). Final distances: S = 0, B = 2, A = 3. The path S-B-A (2+1)
beats the direct S-A (4).
```

Complexity: O(V^2) with a simple array; O(E log V) with a min-heap, since each edge is relaxed once.

Correctness idea: the unsettled vertex with smallest dist cannot be improved later, because all weights are non-negative.

Dijkstra is the standard single-source shortest path algorithm wherever weights are non-negative.

33 Why does Dijkstra's algorithm fail with negative weights?

IMPORTANT

Dijkstra's core promise is: once a vertex is settled, its distance is final and will never improve.

That promise holds only because non-negative weights mean any alternative route through unsettled vertices can only be longer.

A negative edge breaks it: a shorter path to an already settled vertex can appear later via that edge.

Example: S-A weight 5, S-B weight 3, B-A weight -4.

Dijkstra settles A at 5, then discovers S-B-A = -1, but A is already closed.

The true shortest distance to A is -1, which Dijkstra misses - the answer is wrong, not just slow.

Fix: use Bellman-Ford, which relaxes all edges V-1 times and handles negative weights correctly (though not negative cycles).

Settled means final is true only when no edge can pull a distance down afterwards.

34 What is edge relaxation?

IMPORTANT

Relaxation is the single operation at the heart of Dijkstra, Bellman-Ford and Floyd Warshall.

It asks: is the path to v better when routed through u?

```
if dist[u] + w(u, v) < dist[v]: dist[v] = dist[u] + w(u, v)
```

In words: if going source -> ... -> u -> v beats the current best distance to v, update it.

Repeated relaxation propagates improvements outward until no distance can shrink further.

Example: dist[A] = 4, and edge B-A has weight 1 with dist[B] = 2.

Since 2 + 1 = 3 < 4, dist[A] becomes 3.

Every shortest-path algorithm is just relaxation applied in a clever order.